JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent

Class RecursiveTask<V>

java.lang.Object
  java.util.concurrent.ForkJoinTask<V>
    java.util.concurrent.RecursiveTask<V>

All Implemented Interfaces:

Serializable, Future<V>

```
public abstract class RecursiveTask<V>
extends ForkJoinTask<V>
```

A recursive result-bearing ForkJoinTask.

For a classic example, here is a task computing Fibonacci numbers:

```
class Fibonacci extends RecursiveTask<Integer> {
  final int n;
  Fibonacci(int n) { this.n = n; }
  Integer compute() {
    if (n <= 1)
      return n;
    Fibonacci f1 = new Fibonacci(n - 1);
    f1.fork();
    Fibonacci f2 = new Fibonacci(n - 2);
    return f2.compute() + f1.join();
  }
}
```

However, besides being a dumb way to compute Fibonacci functions (there is a simple fast linear algorithm that you'd use in practice), this is likely to perform poorly because the smallest subtasks are too small to be worthwhile splitting up. Instead, as is the case for nearly all fork/join applications, you'd pick some

minimum granularity size (for example 10 here) for which you always sequentially solve rather than subdividing.

Since:

1.7

See Also:

Serialized Form

Constructor Summary

Constructors

Constructor	Description
`RecursiveTask()`	

Method Summary

Abstract Methods | **Concrete Methods**

Modifier and Type	Method	Description
`protected abstract V`	`compute()`	The main computation performed by this task.
`protected boolean`	`exec()`	Implements execution conventions for RecursiveTask.
`V`	`getRawResult()`	Returns the result that would be returned by `ForkJoinTask.join()`, even if this task completed abnormally, or `null` if this task is not known to have been completed.
`protected void`	`setRawResult(V value)`	Forces the given value to be returned as a result.

Methods inherited from class java.util.concurrent.ForkJoinTask

adapt, adapt, adapt, cancel, compareAndSetForkJoinTaskTag, complete, completeExceptionally, fork, get, get, getException, getForkJoinTaskTag, getPool, getQueuedTaskCount, getSurplusQueuedTaskCount, helpQuiesce, inForkJoinPool, invoke, invokeAll, invokeAll, invokeAll, isCancelled, isCompletedAbnormally, isCompletedNormally, isDone, join, peekNextLocalTask, pollNextLocalTask, pollTask, quietlyComplete, quietlyInvoke, quietlyJoin, reinitialize, setForkJoinTaskTag, tryUnfork

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

RecursiveTask

```
public RecursiveTask()
```

Method Detail

compute

```
protected abstract V compute()
```

The main computation performed by this task.

Returns:

the result of the computation

getRawResult

```
public final V getRawResult()
```

Description copied from class: ForkJoinTask

Returns the result that would be returned by `ForkJoinTask.join()`, even if this task completed abnormally, or `null` if this task is not known to have been completed. This method is designed to aid debugging, as well as to support extensions. Its use in any other context is discouraged.

Specified by:

`getRawResult` in class `ForkJoinTask<V>`

Returns:

the result, or null if not completed

setRawResult

```
protected final void setRawResult(V value)
```

Description copied from class: ForkJoinTask

Forces the given value to be returned as a result. This method is designed to support extensions, and should not in general be called otherwise.

Specified by:

`setRawResult` in class `ForkJoinTask<V>`

Parameters:

value - the value

exec

```
protected final boolean exec()
```

Implements execution conventions for RecursiveTask.

Specified by:

`exec` in class `ForkJoinTask<V>`

Returns:

```
true if this task is known to have completed normally
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.